

南開大學
Nankai University

College of Computer Science

NLP - Laboratory Report

HW1: Implementation of Text Classification

Name : Xuanlin Liu (刘轩麟)

Student ID : 2312289

Major : Computer Science

October 7, 2026

Table of Contents

1 Introduction **2**

2 Methods & Implementation **2**

2.1 Bag of Words 2

2.2 Word2Vec 2

2.3 Pre-trained Neural Models 3

2.4 Logistic Regression 3

3 Results & Analysis **3**

3.1 Main Result 3

3.2 Analysis of Performance Differences Among Three Word2Vec Methods 4

3.3 Case Study: The Effect of Truncation Length 4

3.4 Robustness of Binary Bag of Words 5

3.5 Effect of Class Weighting 5

4 Limitations **6**

5 Conclusion **6**

1 Introduction

In this experiment, we train a text classification model, whose crucial component is the text representation method. We adopt three representation approaches: Bag of Words, Word2Vec, and pre-trained neural models. We then present the results obtained with these methods and provide a detailed analysis of them. In addition, we conduct supplementary experiments to investigate the influence of different settings.

2 Methods & Implementation

All representation methods are trained and evaluated on a single, fixed data split of the NYT dataset (80%/10%/10% for training, validation and test, seed = 42), so that the comparison is not confounded by different partitions.

2.1 Bag of Words

Each document is mapped to a sparse vector over the vocabulary of size $|V|$. We consider two variants:

- **Binary Bag of Words:** entry i is 1 if the i-th vocabulary word occurs in the document and 0 otherwise, so only presence is modelled.
- **Word Frequency:** entry i stores the raw count c_i of the i-th word in the document, i.e. $\mathbf{x} = (c_1, c_2, \ldots, c_{|V|})$.

Implementation. Built with scikit-learn's `CountVectorizer`, with tokenisation provided by NLTK's `word_tokenize` . The vocabulary is fitted on the training split only ($|V| = 142{,}222$), while validation and test documents are merely transformed with it to avoid information leakage.

2.2 Word2Vec

A document is represented by the average of the embeddings of its words,

$$e(d) = \frac{1}{n} \sum_{i=1}^{n} e(w_i), \tag{1}$$

where n is the number of in-vocabulary words and out-of-vocabulary words are skipped. All embeddings are 100-dimensional. Three sources of word vectors are compared: (i) the pre-trained GloVe vectors `glove.6B.100d`; (ii) Word2Vec trained on AG News; and (iii) Word2Vec trained on the NYT training split.

Implementation. Self-trained vectors come from Gensim, using the Skip-gram objective with window size 5, `min_count` = 5 and 5 epochs, on AG News (90,000 documents) and on the NYT training split (9,215 documents) respectively. The GloVe vectors are used as published, and document vectors are computed with NumPy.

2.3 Pre-trained Neural Models

We fine-tune bert model with a linear classification head for the three NYT categories.

Implementation. Fine-tuning uses the HuggingFace `transformers` library with a hand-written PyTorch training loop. Sequences are truncated and padded to `max_length` = 64, and training runs for 3 epochs with batch size 32 and AdamW at a learning rate of 2×10^{-5}, on a single NVIDIA RTX 4060 Laptop GPU.

2.4 Logistic Regression

Logistic regression serves as the shared classifier for both the Bag-of-Words and the Word2Vec document vectors, with regularisation strength $C = 1.0$. BERT is excluded here, as it is fine-tuned end-to-end together with its own classification head.

Implementation. Taken directly from scikit-learn's `LogisticRegression`, using the `liblinear` solver for the sparse Bag-of-Words features and `lbfgs` for the dense document vectors. All methods are evaluated on the NYT test split using Accuracy and Macro-F1.

3 Results & Analysis

3.1 Main Result

We report two metrics on the NYT dataset: Accuracy and Macro-F1.

Table 1: Test-set Accuracy and Macro-F1 on the NYT dataset.

Method	Accuracy	Macro-F1
Task 1: Bag of Words		
Binary Bag of Words	0.9896	0.9757
Word Frequency	0.9896	0.9741
Task 2: Word Embedding		
GloVe (pre-trained)	0.9844	0.9646
Word2Vec trained on AG News	0.9757	0.9451
Word2Vec trained on NYT	0.9809	0.9582
Task 3: Pre-trained Neural Models		
Fine-tuned BERT (`bert-base-uncased`)	0.9827	0.9646

Here we could give some basic analysis according to the main results:

- All six representation methods perform strongly, with Accuracy of at least 0.9757 and Macro-F1 of at least 0.9451. This indicates that the three NYT categories are clearly separable at the lexical level and that all three families of representation are adequate for the task.
- The ranking is consistent across both metrics: the two sparse bag-of-words representations rank first, followed by pre-trained GloVe and fine-tuned BERT, and finally the two self-trained Word2Vec variants. The simplest method, Binary Bag of Words with logistic regression, attains the highest scores and slightly exceeds the fine-tuned BERT.

- Binary and frequency-based bag-of-words behave almost identically (both 0.9896 Accuracy). On this dataset the mere **presence** of a word therefore appears to carry nearly all of the signal, while its raw count adds little.

3.2 Analysis of Performance Differences Among Three Word2Vec Methods

The three word-vector methods differ in **training size and domain** (Table 2). GloVe was trained on 6 billion tokens of general text, three orders of magnitude more than either self-trained model, with a vocabulary an order of magnitude larger. Between the self-trained models, the NYT version uses 1.7× more tokens but one tenth as many documents.

The domains also differ: NYT articles average 646 tokens against 39 for AG News snippets, i.e. an AG document is about 17× shorter. Since Word2Vec infers each word vector from its local context, models trained on short snippets transfer less well to NYT classification. This agrees with Table 1: GloVe ranks highest, and among the self-trained models the NYT version outperforms the AG News version.

Table 2: Training corpora of the three word-vector sources. Token counts use the same tokenisation as the classification data.

Source	Documents	Tokens	Tokens/doc	Vocabulary
GloVe 6B (pre-trained)	–	6,000,000,000	–	400,000
Word2Vec trained on NYT	9,215	5,955,677	646	34,753
Word2Vec trained on AG News	90,000	3,477,566	39	26,070

3.3 Case Study: The Effect of Truncation Length

BERT processes at most 64 tokens per document. On the NYT test set the average document contains 646.6 tokens (median 685) and 99.6% of documents exceed 64 tokens, so the model observes on average only 13.7% of a document. Table 3 breaks the test set down by document length.

Table 3: Test error rate by document length. "Seen" is the mean fraction of a document that fits into BERT's 64-token window.

Document length (tokens)	Test docs	Seen by BERT	BoW error rate	BERT error rate
< 200	86	51.2%	2.3%	1.2%
200–400	130	21.0%	0.0%	0.8%
400–700	398	11.4%	0.5%	1.0%
700–1000	471	7.8%	1.5%	2.3%
> 1000	68	5.9%	1.5%	4.4%
All	1153	13.7%	1.0%	1.7%

BERT's error rate rises steadily with document length. It indicates that long documents are generally harder while the fixed window makes them disproportionately hard for BERT.

In the most extreme case, the keywords may not appear before truncation. A single example illustrates the mechanism. The following sports article is classified as *business* by BERT and correctly as *sports* by bag-of-words:

> *seattle — the smoke began billowing innocently from hand-held devices, with colors that were a vague semblance of sounders blue and green. by the time the thick plumes rose into the air above the several thousand marchers, it had combined into a black mixture, not unlike the byproduct of burning coal. red flares helped little to light the way.*

These 64 tokens, which are BERT's entire input, contain no sports vocabulary; the only hint is the proper noun *Sounders*, which is not recognisable as a team name in isolation. The explicitly sporting content appears only afterwards, beginning at token 79 with *"soccer fans it turns out wear scarves for a reason…"*. Bag-of-words sees the full 807-token document, has access to this evidence, and classifies the article correctly.

3.4 Robustness of Binary Bag of Words

Binary bag-of-words represents a document purely as the set of words it contains, discarding word order and context, and scores it by summing an independently learned weight for each word. This simplicity is exactly where its weakness lies. Because *no feature interacts with any other*, the representation is blind to how words combine. It cannot handle negation or qualification, since *"the team did not win"* and *"the team did win"* differ by a single indicator whose weight is small next to those of *team* and *win*; it cannot separate the senses of a word, so *bank* contributes the same evidence whether the article concerns a river or a financial institution; and it cannot capture order, so *government regulation of business* and *business regulation of government* collapse into the same set of words. Every vocabulary item also receives one fixed weight that is estimated essentially on its own, which makes the weights of words occurring only a handful of times in the training data unreliable, while a word never seen during training contributes nothing at all, with no fallback to subwords or characters. A document is thus reduced to a weighted sum over its vocabulary items, in which the meaning of the whole is assumed to be the sum of the meanings of its parts.

3.5 Effect of Class Weighting

The training set is strongly imbalanced (75.4% sports against 12.3% business and 12.2% politics), so it is worth asking whether reweighting the loss by class frequency helps. Table 4 compares the default setting, `class_weight=None`, which is what all the main experiments use, with `class_weight=`balanced'`, which scales each class weight inversely to its frequency.

Table 4: Logistic regression with and without class weighting.

Method	`class_weight`	Accuracy	Macro-F1
Binary BoW	None	0.9896	0.9757
Binary BoW	balanced	0.9879	0.9731
Word Frequency	None	0.9896	0.9741
Word Frequency	balanced	0.9887	0.9729
GloVe	None	0.9844	0.9646
GloVe	balanced	0.9809	0.9581

Class weighting does move the decision boundary in the intended direction: for binary bag-of-words, business recall rises from 0.9270 to 0.9343. The gain is however outweighed by a larger drop

in precision, from 0.9922 to 0.9697, so Macro-F1 falls slightly for all three representations, by 0.0027, 0.0012 and 0.0065 respectively. We therefore retain the default setting, without class weighting, in all main experiments.

4 Limitations

Dataset size. With only 9,215 training and 1,153 test documents, BERT already overfits , and the test set is too small to separate the methods.

Single dataset. All classification experiments use NYT, whose three categories have largely disjoint vocabulary, so the observed ordering need not generalise to other datasets or label granularities.

Single data split. One fixed split (seed = 42) provides no estimate of variance across partitions. Therefore, the generalizability of our results is limited. And the self-trained word vectors vary between runs. It may be more reasonable to conduct multiple runs and report the average results.

No hyperparameter tuning. Each method uses a single first-choice configuration ($C = 1.0$; window 5, `min_count` 5, 5 epochs; learning rate 2×10^{-5}, 3 epochs), so the comparison reflects defaults rather than tuned optima.

5 Conclusion

Through this assignment I gained a much deeper understanding of text representation. I implemented several representation methods, which strengthened my practical skills. Most importantly, I carried out a detailed analysis of the results, including case study, theoretical analysis and supplementary experiments that support the analysis. This has improved my ability to analyse problems and gave me valuable experience in designing experiments.